الجامعة الإسلامية للتكنولوجيا

UNIVERSITÉ ISLAMIQUE DE TECHNOLOGIE

ISLAMIC UNIVERSITY OF TECHNOLOGY

DHAKA, BANGLADESH

ORGANIZATION OF ISLAMIC COOPERATION



CSE 4305

Computer Organization and Architecture

Assignment

Name : Antara Arifa Mullick

Student ID : 210042132

Department : Computer Science and Engineering

Program : B.Sc. in Software Engineering

Semester : Winter 2022-2023 (3rd)

Submission Date : 24/12/2023

Code:

```
addi x13, x13, 10
addi x14, x14, 32
addi x15, x15, 4
addi x16, x16, 30

addi x10, x14, 0
sw x10, 12(x9)

bne x14, x16, ELSE
    addi x13, x13, 3
    addi x14, x14, 20
    add x10, x13, x14
    sw x10, 4(x9)
    j END

ELSE:
    addi x13, x13, 5
    addi x14, x14, 25
    sub x10, x14, x15
    sw x10, 4(x9)

END:
```

Explanation:

1. `addi x13, x13, 10`: Add immediate value 10 to the content of register x13.
2. `addi x14, x14, 32`: Add immediate value 32 to the content of register x14.
3. `addi x15, x15, 4`: Add immediate value 4 to the content of register x15.
4. `addi x16, x16, 30`: Add immediate value 30 to the content of register x16.
5. `addi x10, x14, 0`: Add immediate value 0 to the content of register x14 and store the result in register x10.
6. `sw x10, 12(x9)`: Store the content of register x10 in the memory location at the address obtained by adding 12 to the content of register x9.
7. `bne x14, x16, ELSE`: Branch to the label ELSE if the content of registers x14 and x16 are not equal.

8. `addi x13, x13, 3`: Add immediate value 3 to the content of register x13.
9. `addi x14, x14, 20`: Add immediate value 20 to the content of register x14.
10. `add x10, x13, x14`: Add the content of registers x13 and x14 and store the result in register x10.
11. `sw x10, 4(x9)`: Store the content of register x10 in the memory location at the address obtained by adding 4 to the content of register x9.
12. `j END`: Unconditional jump to the label END.
13. `ELSE:`: Label marking the beginning of the ELSE block.
14. `addi x13, x13, 5`: Add immediate value 5 to the content of register x13.
15. `addi x14, x14, 25`: Add immediate value 25 to the content of register x14.
16. `sub x10, x14, x15`: Subtract the content of register x15 from the content of register x14 and store the result in register x10.
17. `sw x10, 4(x9)`: Store the content of register x10 in the memory location at the address obtained by adding 4 to the content of register x9.
18. `END:`: Label marking the end of the program.

In summary, this assembly code performs a sequence of arithmetic and memory operations, includes conditional branching, and has two blocks of code marked by the labels ELSE and END. The execution flow depends on the comparison between the contents of registers x14 and x16.

Code Snippet:

Input your RISC-V code here:

```
1  addi x13, x13, 10
2  addi x14, x14, 32
3  addi x15, x15, 4
4  addi x16, x16, 30
5
6  addi x10, x14, 0
7  sw x10, 12(x9)
8
9  bne x14, x16, ELSE
10     addi x13, x13, 3
11     addi x14, x14, 20
12     add x10, x13, x14
13     sw x10, 4(x9)
14     j END
15
```

Reset | Stop | CPU: 32 Hz

```
[line 9]: bne x14, x16, ELSE
[line 17]: addi x13, x13, 5
[line 18]: addi x14, x14, 25
[line 19]: sub x10, x14, x15
[line 20]: sw x10, 4(x9)
No more instructions to run! Press Reset to reload the code!
```

Register:

Init Value	Register	Decimal	Hex	Binary
0	x0 (zero)	0	0x00000000	0b00000000000000000000000000000000
0	x1 (ra)	0	0x00000000	0b00000000000000000000000000000000
0	x2 (sp)	0	0x00000000	0b00000000000000000000000000000000
0	x3 (gp)	0	0x00000000	0b00000000000000000000000000000000
0	x4 (tp)	0	0x00000000	0b00000000000000000000000000000000
0	x5 (t0)	0	0x00000000	0b00000000000000000000000000000000
0	x6 (t1)	0	0x00000000	0b00000000000000000000000000000000
0	x7 (t2)	0	0x00000000	0b00000000000000000000000000000000
0	x8 (s0/fp)	0	0x00000000	0b00000000000000000000000000000000
0	x9 (s1)	0	0x00000000	0b00000000000000000000000000000000
0	x10 (a0)	54	0x00000036	0b00000000000000000000000000110110
0	x11 (a1)	0	0x00000000	0b00000000000000000000000000000000
0	x12 (a2)	0	0x00000000	0b00000000000000000000000000000000
5	x13 (a3)	20	0x00000014	0b00000000000000000000000000010100
12	x14 (a4)	69	0x00000045	0b00000000000000000000000001000101
11	x15 (a5)	15	0x0000000f	0b00000000000000000000000000001111
17	x16 (a6)	47	0x0000002f	0b00000000000000000000000000101111
0	x17 (a7)	0	0x00000000	0b00000000000000000000000000000000
0	x18 (s2)	0	0x00000000	0b00000000000000000000000000000000

Memory:

Memory Address	Decimal	Hex	Binary
0x00000000	0	0x00000000	0b00000000000000000000000000000000
0x00000004	54	0x00000036	0b00000000000000000000000000110110
0x00000008	0	0x00000000	0b00000000000000000000000000000000
0x0000000c	44	0x0000002c	0b00000000000000000000000000101100
0x00000010	0	0x00000000	0b00000000000000000000000000000000
0x00000014	0	0x00000000	0b00000000000000000000000000000000
0x00000018	0	0x00000000	0b00000000000000000000000000000000
0x0000001c	0	0x00000000	0b00000000000000000000000000000000
0x00000020	0	0x00000000	0b00000000000000000000000000000000
0x00000024	0	0x00000000	0b00000000000000000000000000000000